

F-CLUSTER — Real ota-server + PostgreSQL system boot (PROVEN, 2026-06-23)

Revision: 1 **Last modified:** 2026-06-23T12:10:00Z **Deliverable:** the foundational real-system enabler — boot the ACTUAL control plane against a REAL PostgreSQL (not the in-memory store) so every higher test type (integration/e2e/full-automation/security/stress/chaos, §11.4.27) hits ONE live instance. Verdict: **PASS** with captured evidence. Secrets redacted (§11.4.10).

What runs

- `server/deploy/system.compose.yml` — rootless-podman stack (§11.4.161): `postgres:16-alpine` + `ota-control-plane` built from a cross-compiled static `linux/amd64 ota-server`. No privileged, no host networking, no root.
- `tests/lib/boot_real_system.sh` — the §11.4.76 on-demand boot harness: cross-compile the server on the host → rsyncs a minimal bundle → `podman-compose build` → ordered up (`postgres` → `pg_isready` wait → `ota-server`) → waits `/readyz`→200 → prints the live base URL. `--down` tears down + cleans (project-scoped, leaves the integration suite's `postgres` untouched).
- Driven from this host as `milosvasic` over SSH to `thinker.local` (rootless podman 4.9.3). No sudo.

Captured proof (docs/qa/20260623-real-system-boot/)

- `boot_run3.log` — PG-WAIT: `postgres` accepting connections → **READY: /readyz -> 200 {"status":"ready"}**; both containers Up (healthy).
- `api_smoke.txt` — real DB-backed API smoke against the live system:
 - `/readyz` → `{"status":"ready"}` [HTTP 200] (liveness)
 - `/api/v1/devices` (no auth) → `{"error":{"code":"UNAUTHENTICATED",...,"request_id":...}}` [HTTP 401] (router + auth middleware live)
 - `POST /api/v1/auth/login` (admin) → [HTTP 200] real JWT `access_token` + `refresh_token` + `roles: [admin,operator,viewer]` (full **DB-backed auth path** — token redacted §11.4.10)

A green `/readyz` alone is shallow liveness; a real signed token from a real login proves the *system* works end-to-end against the real DB (§11.4.107-grade).

Two real fixes this required (§11.4.102 root-cause-first)

1. **Server startup retry** (`server/cmd/ota-server/main.go`) — the server did `log.Fatalf` on the *first* postgres ping; a freshly-started postgres reports its container “up” before accepting connections, so the server `Exited (1)` on connection refused (a boot-ordering race that bites `compose/k8s/systemd` deploys). Fix: bounded retry-with-backoff (up to 60s) at startup — production-correct robustness, not a test patch.
2. **Harness clean-slate** (`boot_real_system.sh`) — a re-run collided: `podman-compose`’s internal `recreate` can’t drop a postgres a leftover `ota-server` still depends on (exit 125, name-in-use). Fix: force-clean by project label (`do_down`) at the start of `do_up`.

RED (pre-fix): `/readyz HTTP 000, ota-server Exited (1) connection refused`. GREEN (post-fix): `/readyz 200 + DB-backed login`. Both captured.

Honest boundary (§11.4.6)

This proves the real OTA system boots + serves real DB-backed requests on one rootless-podman host (thinker = the live “cluster” node). It is NOT a multi-node cluster (still aspirational for OTA, §11.4.69-cluster gap G2). The stack was torn down after capture (§11.4.14); higher suites re-boot it on demand via the harness, which prints the live base URL as its single contract.